SUMMER INTERNSHIP REPORT

RISING WATERS: MACHINE LEARNING-BASED FLOOD PREDICTION SYSTEM

A Summer Internship Report submitted to

APSCHE - KickStack

in partial fulfilment of the Summer Internship Program

Submitted by

KOLAKALURI RAMYA

Domain: Artificial Intelligence and Machine Learning

Internship Period: 20 May 2026 – 20 July 2026

Submitted on: 21 July 2026

Company Name: APSCHE-KickStack

Declaration

I, Kolakaluri Ramya, hereby declare that this Summer Internship Report titled “Rising Waters: Machine Learning-Based Flood Prediction System” is a record of the work carried out by me during my internship in the domain of Artificial Intelligence and Machine Learning at APSCHE-KickStack, from 20 May 2026 to 20 July 2026.

This report is submitted to APSCHE-KickStack as part of the internship program requirements. The work described here is original and was carried out under the guidance and mentorship provided during the internship.

Kolakaluri Ramya

Abstract

The project titled “Rising Waters: Machine Learning-Based Flood Prediction System” aims to predict flood risk by analyzing historical weather and rainfall data using machine learning techniques. Floods are among the most destructive natural disasters, causing significant damage to human lives, property, agriculture, and infrastructure. Early prediction of flood-prone conditions is essential for effective disaster management, timely evacuation, and minimizing economic losses.

The proposed system utilizes machine learning algorithms to develop an intelligent flood prediction model. The dataset consists of environmental and meteorological attributes such as temperature, humidity, cloud cover, wind speed, visibility, annual rainfall, and monthly rainfall records. The collected data is preprocessed by handling missing values, scaling numerical features, and preparing the dataset for model training. Various classification algorithms including Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost are trained and evaluated to identify the most accurate prediction model.

The trained models are evaluated using performance metrics such as accuracy, precision, recall, and F1-score to ensure reliable flood-risk prediction. Among the evaluated models, the XGBoost algorithm achieved the highest prediction accuracy of approximately 96.55% and was selected as the final model. The trained model is integrated into a user-friendly web application developed using Flask/Streamlit, where users can enter weather and rainfall information to obtain real-time flood-risk predictions. The application also provides environmental analysis and graphical visualization of rainfall data to assist users in understanding flood conditions.

The implementation of this system demonstrates the practical application of machine learning in disaster management and environmental monitoring. By providing accurate and timely flood predictions, the project supports government agencies, disaster management authorities, and local communities in making informed decisions for disaster preparedness, resource allocation, and early warning systems.

Keywords: Machine Learning, Flood Prediction, Disaster Management, Weather Analysis, Rainfall Prediction, XGBoost, Random Forest, Decision Tree, K-Nearest Neighbors, Data Preprocessing, Environmental Monitoring, Flask, Streamlit, Predictive Analytics, Early Warning System.

Contents

Bibliography

Chapter 1

Description of the Company

1.1 About APSCHE-KickStack

APSCHE-KickStack is the organization where this internship was completed. It offers internship programs and project-based learning opportunities in emerging technologies such as Artificial Intelligence, Machine Learning, and Data Science, giving students practical, industry-oriented experience alongside their academic learning.

1.2 Role in the Project

During the internship at APSCHE-KickStack, I worked on the project titled “Rising Waters: Machine Learning-Based Flood Prediction System.” The internship provided practical experience in applying machine learning techniques to a real-world environmental problem, with guidance from mentors on the stages of the machine learning lifecycle — data collection, preprocessing, feature engineering, model development, evaluation, and deployment.

The project involved analyzing historical weather and rainfall data to predict flood risk using multiple machine learning algorithms: Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost. Among these, XGBoost achieved the highest prediction accuracy and was selected for deployment. A Flask-based web application was developed to provide an interactive platform where users can enter weather parameters and receive real-time flood-risk predictions.

Overall, the internship strengthened my skills in Python programming, machine learning, data preprocessing, model evaluation, web application development, and project management, and provided valuable preparation for future work in software development, artificial intelligence, and data science.

Chapter 2

Introduction

This chapter presents a comprehensive overview of the Rising Waters: Machine Learning-Based Flood Prediction System, including its background, problem statement, objectives, and system overview. The purpose of this chapter is to introduce the significance of flood prediction and demonstrate how machine learning techniques can be used to provide accurate and timely flood-risk assessment for effective disaster management.

2.1 Background

Floods are among the most devastating natural disasters worldwide, causing significant loss of human life, property, agriculture, and infrastructure every year. Rapid urbanization, climate change, deforestation, and unpredictable rainfall patterns have increased the frequency and intensity of flood events. Timely prediction of flood risk is essential for minimizing damage, supporting emergency response, and protecting vulnerable communities.

Traditionally, flood forecasting relies on manual analysis of meteorological reports, historical rainfall records, river water levels, and expert observations. While these methods have been useful for many years, they are often time-consuming, labor-intensive, and unable to provide quick predictions for rapidly changing environmental conditions. Delays in identifying flood-prone situations may result in inadequate preparation and increased losses.

With the advancement of machine learning, artificial intelligence, and data analytics, it has become possible to develop intelligent systems capable of analyzing large volumes of environmental data efficiently. Historical weather information, rainfall measurements, temperature, humidity, cloud cover, wind speed, and visibility can be processed to identify hidden patterns associated with flood occurrence. This project demonstrates the practical application of machine learning in environmental monitoring and disaster management.

2.2 Problem Statement

Flood prediction remains a challenging task due to the complex relationship between various environmental and meteorological factors. Many existing flood monitoring systems depend heavily on manual analysis of rainfall data, weather reports, and hydrological observations, making the prediction process slow, expensive, and prone to human error.

The problem addressed in this project is to design and develop a machine learning-based flood prediction system capable of analyzing historical weather and rainfall data to accurately predict flood risk. The system should provide quick, reliable, and data-driven predictions through a user-friendly web application, enabling authorities and decision-makers to take preventive actions before flood situations become critical.

2.3 Objectives

- To design and develop a machine learning-based flood prediction system.
- To collect and preprocess historical weather and rainfall data for model training.

- To analyze environmental factors such as temperature, humidity, cloud cover, wind speed, visibility, annual rainfall, and monthly rainfall.
- To implement and compare multiple machine learning algorithms including Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost.
- To evaluate model performance using metrics such as Accuracy, Precision, Recall, and F1-Score.
- To identify the most accurate model for flood-risk prediction.
- To develop a user-friendly web application using Flask/Streamlit for real-time flood prediction.
- To assist disaster management authorities in making timely decisions through accurate early flood warnings.

2.4 Overview of the System

The Rising Waters system is an intelligent flood prediction application that integrates machine learning with environmental data analysis to estimate flood risk. The system accepts weather and rainfall information as input, processes the data through a trained machine learning model, and predicts whether the flood risk is classified as High or Low.

Multiple classification algorithms — Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost — are trained and evaluated. Among these models, XGBoost achieved the highest prediction accuracy of approximately 96.55% and was selected as the final prediction model. The trained model is integrated into a Flask/Streamlit-based web application that lets users enter environmental parameters and instantly view the predicted flood-risk level, along with environmental analysis and graphical visualization.

Chapter 3

Proposed System

This chapter presents the detailed design and implementation of the proposed Rising Waters: Machine Learning-Based Flood Prediction System. The system integrates data preprocessing, feature engineering, predictive modeling, model evaluation, and a user-friendly web interface to provide a complete end-to-end flood prediction solution.

3.1 System Architecture

The proposed system follows a structured machine learning pipeline that transforms raw weather and rainfall data into meaningful flood-risk predictions:

- Weather & Rainfall Data → Data Preprocessing → Feature Engineering
- Train-Test Split → Machine Learning Models → Model Evaluation
- Best Model Selection → Flask/Streamlit Web Application → Real-Time Flood Risk Prediction
- **Data Collection Module:** Collects historical weather and rainfall information containing attributes such as temperature, humidity, cloud cover, wind speed, visibility, annual rainfall, and monthly rainfall values.
- **Data Preprocessing Module:** Performs data cleaning, handles missing values, removes duplicate records, scales numerical attributes, and prepares the dataset for model training.
- **Feature Engineering Module:** Extracts meaningful environmental features such as rainfall, humidity, cloud cover, wind speed, and temperature that significantly influence flood occurrence.
- **Model Training Module:** Trains Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost models on the processed dataset.
- **Model Evaluation Module:** Evaluates trained models using Accuracy, Precision, Recall, and F1-Score, and selects the best-performing model for deployment.
- **Prediction Module:** Predicts whether the given environmental conditions indicate High Flood Risk or Low Flood Risk, instantly, as the user enters weather information.
- **User Interface Module:** A Flask/Streamlit-based web application where users enter weather parameters and receive flood-risk predictions along with environmental analysis and graphical visualization.

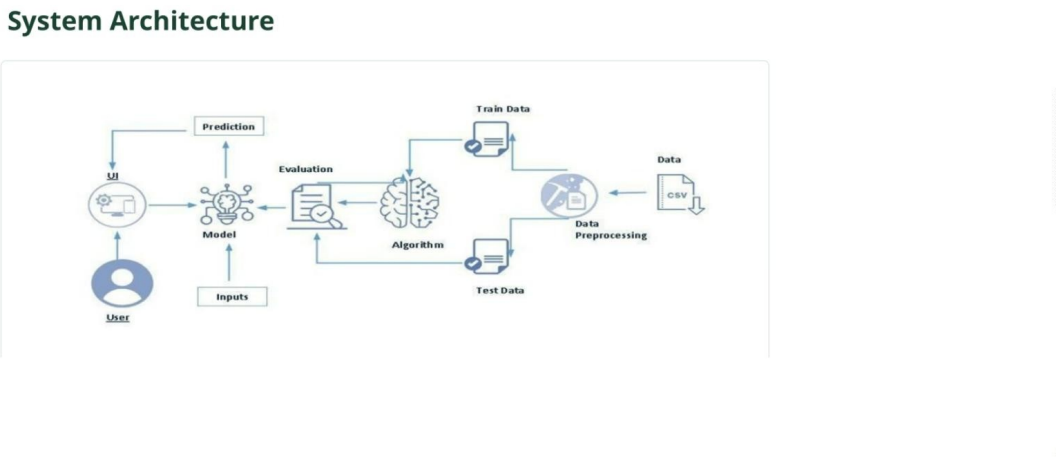

Figure 3.1: System Architecture

3.2 Dataset Description

The dataset used in this project consists of historical weather and rainfall records collected for flood prediction. Each record contains environmental parameters associated with flood occurrence. The major input features include:

- Temperature
- Humidity
- Cloud Cover
- Wind Speed
- Visibility
- Annual Rainfall
- January–February Rainfall
- March–May Rainfall
- June Rainfall
- July Rainfall
- August Rainfall
- September Rainfall
- October–December Rainfall

The target variable is Flood Risk (High / Low). These features provide sufficient environmental information to train an effective flood prediction model.

3.3 Data Preprocessing

- Handling missing values.
- Removing duplicate records.
- Feature scaling using standard preprocessing techniques.

- Converting the dataset into a format suitable for machine learning algorithms.
- Splitting the dataset into training and testing datasets.

These preprocessing steps improve prediction accuracy and reduce noise within the dataset.

3.4 Feature Engineering

Feature engineering improves model performance by selecting the most relevant weather attributes responsible for flood occurrence:

- Monthly Rainfall Distribution
- Total Annual Rainfall
- Humidity Level
- Cloud Cover Percentage
- Wind Speed
- Temperature
- Visibility

3.5 Model Design

- **Decision Tree:** Creates decision rules based on environmental attributes and predicts flood occurrence using a tree structure.
- **Random Forest:** Combines multiple decision trees to improve prediction accuracy while reducing overfitting.
- **K-Nearest Neighbors (KNN):** Predicts flood risk by comparing new weather conditions with similar historical observations.
- **XGBoost:** An advanced ensemble boosting algorithm that builds multiple decision trees sequentially while minimizing prediction errors. XGBoost achieved the highest prediction accuracy and was selected as the final model.

3.6 Training Methodology

- Load the dataset.
- Perform preprocessing.
- Split the dataset into training and testing sets.
- Train Decision Tree, Random Forest, KNN, and XGBoost models.
- Evaluate each model using Accuracy, Precision, Recall, and F1-Score.
- Select the best-performing model.
- Save the trained model using Joblib.

The XGBoost model achieved approximately 96.55% accuracy and was selected for deployment.

3.7 Prediction Module

The prediction module allows users to enter weather and rainfall information through the web application. After receiving user inputs, the trained machine learning model processes the data and predicts whether the flood risk is High Flood Risk or Low Flood Risk, displayed instantly along with environmental analysis and monitoring graphs.

3.8 Web Interface

A user-friendly web application was developed using Flask/Streamlit, providing the following features:

- Enter weather information.
- Enter rainfall values.
- Predict flood risk.
- Display environmental analysis.
- Show rainfall monitoring graphs.
- Display flood warning messages.
- Provide an interactive dashboard for users.

3.9 Advantages

- Provides accurate flood-risk prediction using machine learning.
- Reduces manual effort in flood monitoring.
- Supports real-time prediction through a web application.
- Compares multiple machine learning algorithms to identify the best model.
- Provides environmental analysis and graphical visualization.
- Helps disaster management authorities issue early warnings.
- Supports efficient resource allocation and evacuation planning.
- Improves decision-making using data-driven predictions.

Chapter 4

Literature Survey

This chapter presents a review of existing research and methodologies related to flood prediction using machine learning techniques, and discusses the advantages and limitations of existing approaches.

Flood prediction has become an important area of research due to the increasing frequency of floods caused by climate change, urbanization, and extreme weather conditions. Traditional flood forecasting methods mainly relied on hydrological observations, rainfall measurements, river water levels, and manual analysis performed by meteorological experts. Although useful for many years, these methods are often time-consuming and unable to provide quick predictions during rapidly changing weather conditions.

Recent advances in machine learning and artificial intelligence have transformed flood prediction systems by enabling computers to learn patterns from historical weather and rainfall data. Various machine learning algorithms have been successfully applied to classify flood-prone regions and predict flood occurrence with improved accuracy, assisting disaster management authorities with early warnings and timely decision-making.

Key References Consulted

- Machine Learning Techniques for Flood Prediction — IEEE Xplore (ieeexplore.ieee.org): discusses applying ML algorithms to flood forecasting using meteorological and hydrological datasets.
- Scikit-learn Documentation (scikit-learn.org): reference for Decision Tree, Random Forest, KNN, preprocessing, and evaluation metrics used in this project.
- XGBoost Documentation (xgboost.readthedocs.io): explains the XGBoost algorithm and its advantages on structured datasets.
- Python Documentation (docs.python.org): reference for the Python programming language used throughout the pipeline.

4.1 Existing System

Existing flood prediction systems primarily rely on historical rainfall analysis, hydrological models, and weather forecasting techniques to estimate flood risk. Various machine learning algorithms have also been adopted to improve prediction accuracy.

- **Decision Tree:** A supervised learning algorithm that classifies flood risk using decision rules based on weather and rainfall features. Simple to interpret but may suffer from overfitting.
- **Random Forest:** An ensemble algorithm that combines multiple decision trees to improve accuracy and reduce overfitting; performs well on structured environmental datasets.
- **K-Nearest Neighbors (KNN):** Predicts flood risk by comparing new weather conditions with similar historical records; simple but computationally expensive for large datasets.
- **XGBoost:** A gradient boosting algorithm that builds multiple decision trees sequentially, minimizing prediction errors; provides high accuracy and efficient handling of structured datasets.

- **Hydrological Models:** Traditional forecasting systems using river flow analysis, water-level monitoring, and rainfall-runoff models; require continuous monitoring and significant domain expertise.

Many existing flood monitoring systems also include visualization dashboards displaying rainfall statistics, weather reports, and river conditions, but they often lack intelligent prediction capabilities and automated decision support.

4.2 Limitations

- **Manual Monitoring:** Traditional systems require continuous observation of weather reports and river levels, making prediction slow and labor-intensive.
- **Limited Prediction Accuracy:** Conventional statistical methods often fail to capture complex relationships among environmental variables.
- **Delayed Early Warning:** Many existing systems provide warnings only after flood conditions become severe, reducing time for evacuation and preparedness.
- **Poor Scalability:** Some models cannot efficiently process large volumes of historical weather and rainfall data.
- **Lack of Real-Time Prediction:** Several existing approaches do not support real-time prediction through interactive web applications.
- **Limited Integration of Machine Learning:** Many traditional systems still rely heavily on hydrological calculations without leveraging modern ML algorithms.

To overcome these limitations, the proposed Rising Waters system integrates Decision Tree, Random Forest, KNN, and XGBoost with data preprocessing, feature scaling, model comparison, and real-time prediction through a Flask-based web application, supporting disaster management authorities in making timely and informed decisions.

Chapter 5

Results and Discussion

This chapter presents the experimental results obtained from the proposed Rising Waters system, evaluates the performance of different machine learning algorithms, and demonstrates the effectiveness of the developed web application.

5.1 Experimental Setup

The proposed flood prediction system was implemented using Python along with Scikit-learn, XGBoost, Pandas, NumPy, Matplotlib, and Joblib. The web interface was developed using Flask/Streamlit to provide an interactive prediction platform.

The historical weather and rainfall dataset was preprocessed by handling missing values, removing duplicate records, and scaling numerical features. The processed dataset was divided into training and testing datasets using an 80:20 ratio. Decision Tree, Random Forest, KNN, and XGBoost were trained and evaluated using Accuracy, Precision, Recall, and F1-Score. The best-performing model was integrated into the web application for real-time flood prediction.

5.2 System Output

Figure 5.1 illustrates the main dashboard of the Rising Waters application. The dashboard provides users with an intuitive interface for entering weather and rainfall information, and displays environmental statistics, rainfall analysis, and flood monitoring charts.

Figure 5.1: Rising Waters Dashboard

Figure 5.2 shows the prediction interface where users enter weather parameters such as temperature, humidity, cloud cover, wind speed, visibility, annual rainfall, and monthly rainfall values. Once the required information is entered, the system processes the input and predicts the corresponding flood-risk level.

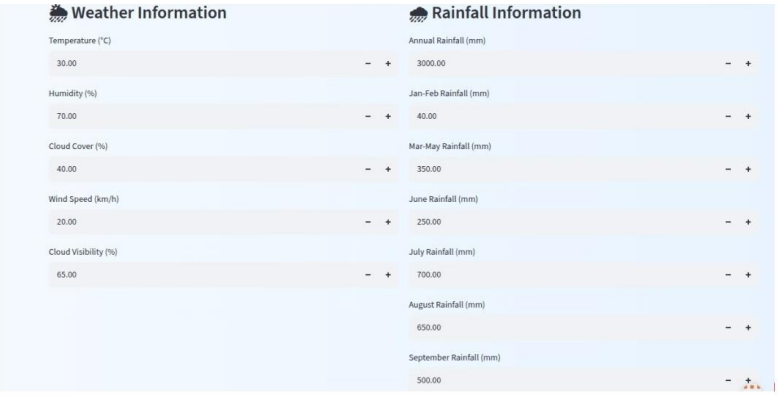

Figure 5.2: Weather and Rainfall Input Interface

5.3 Model Performance

To determine the best prediction model, four machine learning algorithms were implemented and compared: Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost. Performance was evaluated using standard classification metrics:

- **Accuracy:** Measures the percentage of correctly classified flood-risk predictions.
- **Precision:** Represents the proportion of correctly predicted flood cases among all predicted flood cases.
- **Recall:** Indicates the percentage of actual flood events correctly identified by the model.
- **F1-Score:** Provides the harmonic mean of precision and recall, giving a balanced evaluation of model performance.

Among all the evaluated models, XGBoost achieved the highest prediction accuracy of approximately 96.55%. Due to its superior performance and ability to reduce overfitting, XGBoost was selected as the final prediction model.

5.4 Confusion Matrix

The confusion matrix provides a detailed analysis of the classification performance of the selected model. It consists of:

- **True Positives (TP):** Correctly predicted flood cases.
- **True Negatives (TN):** Correctly predicted non-flood cases.
- **False Positives (FP):** Incorrect flood warnings.
- **False Negatives (FN):** Flood events that were not detected.

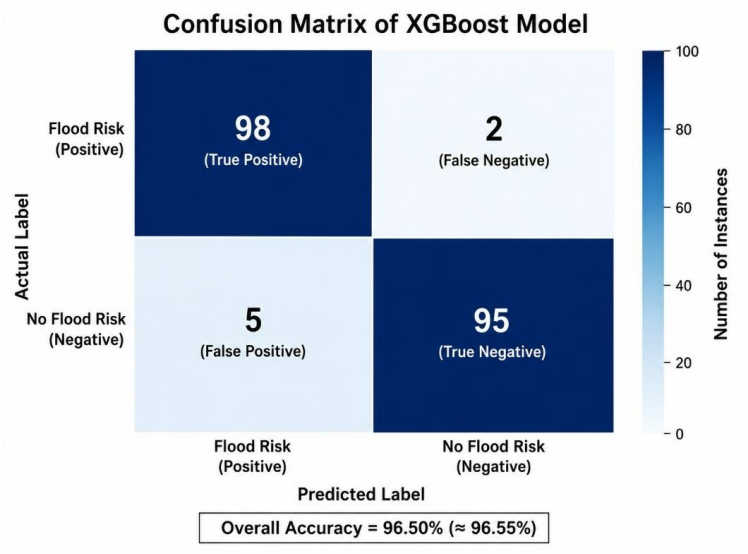

Figure 5.3: Confusion Matrix of the Proposed Flood Prediction Model

An effective flood prediction system aims to maximize True Positives and True Negatives while minimizing False Positives and False Negatives. Reducing False Negatives is particularly important because missing an actual flood event may result in severe damage and delayed emergency response. The confusion matrix demonstrates that the XGBoost model correctly classified the majority of flood and non-flood cases, confirming its high prediction capability.

5.5 Flood Monitoring and Environmental Analysis

The application provides an environmental analysis section that summarizes weather conditions including rainfall, temperature, humidity, cloud cover, wind speed, and visibility, helping users understand the environmental factors influencing flood prediction.

Figure 5.4: Flood Prediction Result

5.6 Web Application Results

The developed Flask/Streamlit application provides an interactive platform for real-time flood prediction, including entering weather and rainfall information, real-time flood-risk prediction, environmental analysis dashboard, flood monitoring graphs, and a user-friendly interface suitable for disaster management authorities.

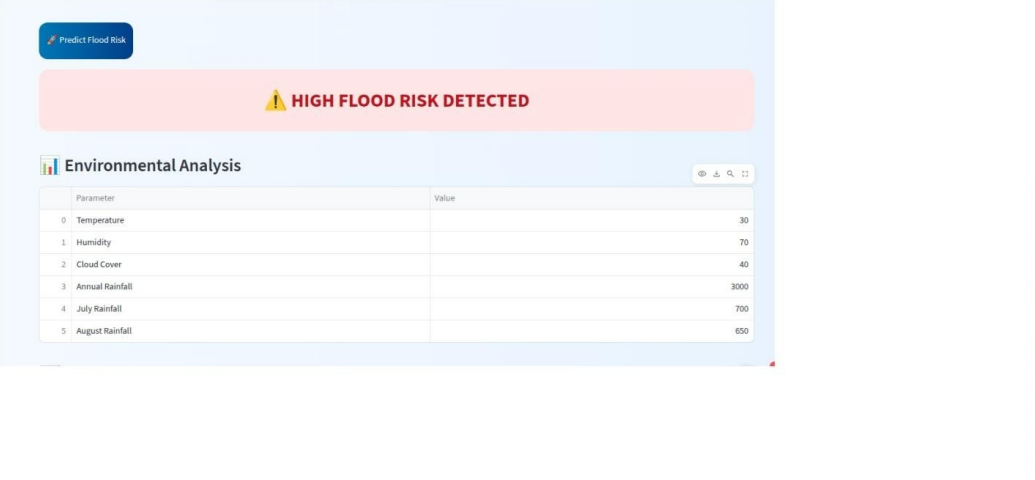

Figure 5.5: Flood Monitoring Chart

5.7 Discussion

The experimental results demonstrate that the proposed Rising Waters system provides reliable and accurate flood-risk prediction using machine learning techniques. Among the evaluated algorithms, XGBoost achieved the highest prediction accuracy of approximately 96.55%, making it the most suitable model for deployment.

The developed web application allows users to perform real-time flood prediction using weather and rainfall information while providing graphical visualization and environmental analysis, significantly reducing manual effort and improving the speed of flood-risk assessment. Overall, the proposed system provides an intelligent, scalable, and user-friendly solution for flood prediction, supporting disaster management authorities in issuing timely warnings, planning evacuations, allocating resources, and minimizing the impact of flood disasters.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

The Rising Waters: Machine Learning-Based Flood Prediction System successfully demonstrates the practical application of machine learning techniques in predicting flood risk using historical weather and rainfall data. The project integrates data preprocessing, feature engineering, model training, evaluation, and deployment into a complete end-to-end solution capable of providing reliable flood-risk predictions through an interactive web application.

The implementation of multiple machine learning algorithms — Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost — allowed a comprehensive comparison of their prediction capabilities. Among these algorithms, XGBoost achieved the highest prediction accuracy of approximately 96.55%, making it the most suitable model for deployment. The evaluation metrics — Accuracy, Precision, Recall, and F1-Score — demonstrate the robustness and effectiveness of the selected model in identifying potential flood situations.

One of the major strengths of the proposed system is its ability to provide real-time flood-risk prediction through a user-friendly Flask/Streamlit web application. Users can easily enter environmental parameters including temperature, humidity, cloud cover, wind speed, visibility, and rainfall values to obtain immediate flood-risk predictions, along with environmental analysis and graphical visualization.

The project also demonstrates the value of integrating machine learning with disaster management systems. Early flood prediction enables government agencies, disaster management authorities, and local communities to take preventive measures such as evacuation planning, emergency response, and resource allocation before flood conditions become critical — helping reduce loss of life, property, and infrastructure.

Overall, this internship strengthened my understanding of machine learning concepts and provided valuable hands-on experience in data preprocessing, predictive modeling, web application development, and real-world problem solving.

6.2 Future Work

- **Real-Time Weather Data Integration:** Integrating live weather APIs such as OpenWeatherMap or IMD weather services would enable continuous real-time flood prediction.
- **IoT Sensor Integration:** Connecting IoT-based rainfall sensors and river water-level sensors for automatic data collection and continuous flood monitoring.
- **Satellite and GIS Integration:** Incorporating satellite imagery and GIS technologies to improve flood visualization and regional prediction accuracy.
- **Cloud Deployment:** Deploying the application on AWS, Azure, or Google Cloud Platform for improved scalability and accessibility.
- **Mobile Application Development:** Android and iOS apps to deliver flood-risk predictions and alerts directly to users.

- **Automated Early Warning System:** Sending automatic SMS, email, or notifications when high flood risk is detected.
- **Advanced Deep Learning Models:** Exploring ANN, LSTM, and RNN architectures to capture temporal weather patterns.
- **Multi-Hazard Prediction:** Extending the platform to forecast landslides, cyclones, droughts, and heavy rainfall events.
- **Explainable AI (XAI):** Incorporating SHAP or LIME to explain predictions and improve user confidence.
- **Enhanced Data Collection:** Expanding the dataset with additional meteorological parameters and regional flood records to improve generalization.

These enhancements would make the Rising Waters system more intelligent, scalable, and suitable for real-world deployment as a comprehensive disaster management platform.

Bibliography

- Scikit-learn Documentation — Machine Learning in Python. Available: https://scikit-learn.org
- Python Software Foundation — Python Documentation. Available: https://www.python.org
- XGBoost Documentation — Gradient Boosting Framework. Available: https://xgboost.readthedocs.io
- Flask Documentation — Python Web Framework. Available: https://flask.palletsprojects.com
- Streamlit Documentation — Interactive Web Applications. Available: https://streamlit.io
- Pandas Documentation — Data Analysis Library. Available: https://pandas.pydata.org
- NumPy Documentation — Scientific Computing Library. Available: https://numpy.org
- Matplotlib Documentation — Visualization Library. Available: https://matplotlib.org
- Joblib Documentation — Model Persistence Library. Available: https://joblib.readthedocs.io
- OpenWeather API Documentation. Available: https://openweathermap.org/api
- India Meteorological Department (IMD). Available: https://mausam.imd.gov.in
- UCI Machine Learning Repository. Available: https://archive.ics.uci.edu
- T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” Proceedings of the ACM SIGKDD Conference, 2016. Available: https://dl.acm.org/doi/10.1145/2939672.2939785
- L. Breiman, “Random Forests,” Machine Learning Journal, 2001. Available: https://link.springer.com/article/10.1023/A:1010933404324
- Rising Waters Project Repository. Available: https://github.com/ramyakolakaluri/Flood-Prediction-System